

WhiteDavid23 Partner Portal — System Design Document

Version: 1.0 **Prepared for:** Internship submission (Step 24 — Documentation) **Derived from:** Software Requirements Specification v1.0, Developer Handover Pack v1.0 **Companion to:** 24-step build log (Steps 1-24)

Table of Contents

1. Purpose & Scope
2. Requirements Reconciliation
3. High-Level Design (HLD)
4. Low-Level Design (LLD)
5. Frontend Design
6. Backend Design
7. Database Design
8. Core Workflows
9. Security Design
10. Non-Functional Requirements Mapping
11. Deployment Architecture
12. Documentation & Reference Stack
13. Appendix — Phase Plan Alignment

1. Purpose & Scope

This document translates the business-level requirements in the SRS and Developer Handover Pack into a concrete technical design: the architecture, module boundaries, data models, and workflows that the codebase (built step-by-step across Steps 1-24) actually implements.

In scope: role-based CRM covering leads, follow-ups, admissions, courses, partners, commissions, notifications, audit logs, reports, and account settings — matching the SRS's Phase 1 + Phase 2 MVP priority list.

Out of scope (Phase 3+ per SRS §13 / Handover §14): WhatsApp integration, payment gateway automation, AI lead scoring, LMS/certificate features, franchise support. These are noted where relevant so the architecture doesn't block them later, but none are built in this pass.

2. Requirements Reconciliation

The SRS and Handover Pack are business documents; a few decisions were needed to turn them into a buildable system. This table records those decisions so the "why" isn't lost.

Requirement (source)	Decision made	Rationale
4 roles: Super Admin, Partner Admin, Team Member, Support (SRS §3, Handover §4)	Implemented exactly as <code>super_admin</code> , <code>partner_admin</code> , <code>team_member</code> , <code>support</code> enum on the User model	Matches both docs verbatim; no ambiguity
"Lead scoring" (SRS §5.4)	Deferred — not built in Phase 1/2	SRS itself lists AI lead scoring under Future Enhancements (§13) and Handover Phase 4; the "scoring" mentioned in §5.4 is inconsistent with that — treated as forward-looking, not MVP
"Two-factor authentication" (SRS §5.1)	Deferred	SRS itself says "future releases" in the same sentence
Database: MongoDB or PostgreSQL (both docs)	MongoDB chosen	Document model fits the varied per-role data shape (e.g. <code>partnerOrg</code> nullable, flexible <code>marketingAssets</code> array) better than a rigid relational schema at MVP stage; matches Handover's own stack recommendation order
"Account Settings" module (Handover §5, screen list)	Built as Step 10, folded into Dashboard step rather than a standalone numbered step	The 24-step plan didn't originally have a dedicated step; explicitly reconciled and added rather than silently dropped
Marketing Center (SRS §5.8)	Modeled as <code>marketingAssets[]</code> embedded in the <code>Course</code> document, not a separate top-level entity	SRS's own data model (§6.1) doesn't list "Creative Asset" as needing independent lifecycle/ownership beyond a course; embedding avoids an unnecessary join for what is Phase 2 scope
Campaign Center (SRS §5.9)	Not built	SRS explicitly marks this "in future releases"

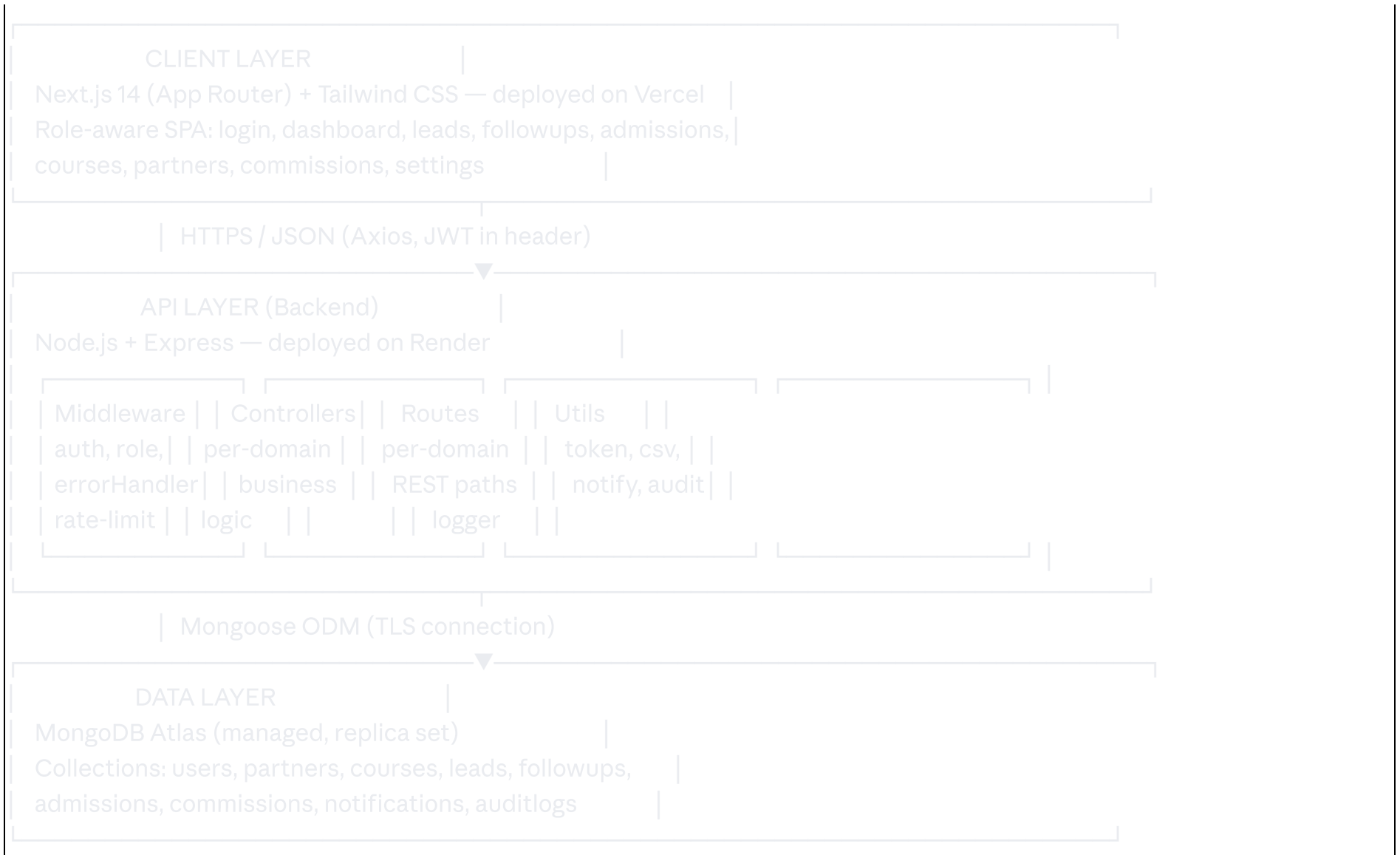
Requirement (source)	Decision made	Rationale
"Lead Activity" / "Call logs" / "WhatsApp logs" as separate entities (SRS §6.1, Handover §5)	Unified into <code>notes</code> + <code>FollowUp</code> documents rather than a separate outcome + timestamp); WhatsApp/call channel tracking is Activity/Log collection	Avoids overlapping entities (FollowUp already carries Phase 3 (external integration)

3. High-Level Design (HLD)

3.1 Architecture Style

A conventional **three-tier client-server architecture**: a Next.js SPA-style frontend, a stateless REST API backend, and a managed document database. This is a deliberate simplification relative to the SRS's mention of future microservices-adjacent scale (§9.5) — at MVP scale a modular monolith is faster to build, easier to reason about for a solo/small team, and the module boundaries (controllers/routes per domain) are already drawn so a future service-extraction is possible without a rewrite.





3.2 Component Responsibilities

Layer	Responsibility	Does NOT do
Frontend Rendering, client-side route protection (<code>RequireAuth</code>), form		Business rules (e.g. duplicate-lead detection),

Layer	Responsibility	Does NOT do
	validation UX, calling the API	authorization decisions (only <i>hides</i> UI; the API is the real gate)
Backend	All business logic, authorization enforcement, data validation, commission calculation, audit logging	Rendering, session/UI state
Database	Durable storage, uniqueness constraints (e.g. <code>email</code> unique index), query performance (indexes on <code>partnerOrg</code> , <code>normalizedPhone</code>)	Business logic (kept in the application layer, not stored procedures)

3.3 Why This Stack (per SRS Appendix §16.2 / Handover §17)

Both source documents independently converge on the same recommended stack (Next.js/React + Tailwind, Node/Express, MongoDB/PostgreSQL, JWT+RBAC, Vercel+Render/AWS). This design follows that recommendation directly rather than deviating — there's no requirement in either document that argues for a different stack, and matching it keeps the build inside what both stakeholders already expect.

4. Low-Level Design (LLD)

4.1 Backend Module Map

Each domain gets the same four-file shape — this consistency is intentional so a developer who understands one module (e.g. Leads) can navigate any other (Admissions, Courses, Partners) without relearning conventions.



models/<Domain>.js → Mongoose schema (shape, validation, indexes, pre-save hooks)
 controllers/<domain>Controller.js → business logic, role-scoped queries, calls to utils
 routes/<domain>Routes.js → Express router: maps HTTP verb+path → middleware chain → controller

Every protected route follows the same middleware chain:

```

  router.<verb>('/path', protect, authorize(...roles), controllerFn)
    |
    |
    | ─ optional: rejects if req.user.role not in list
    | ─ verifies JWT, attaches req.user
    ─ Express router method
  
```

4.2 Domain Module Table

Domain	Model	Controller	Routes	Role scope function
Auth	User	authController	authRoutes	n/a (self-service: register/login/me)
Leads	Lead	leadController	leadRoutes	buildScopeFilter — super_admin: all; partner_admin: org; team_member: assigned only
Follow-ups	FollowUp	followUpController	followUpRoutes	same pattern as Leads
Admissions	Admission	admissionController	admissionRoutes	super_admin: all; others: own org
Courses	Course	courseController	courseRoutes	catalog is global-read; write restricted to super_admin
Partners	Partner	partnerController	partnerRoutes	super_admin: all; others: own org only

Domain	Model	Controller	Routes	Role scope function
Commissions	<code>Commission</code>	<code>commissionController</code>	<code>commissionRoutes</code>	same org-scoping as Admissions; payout status change restricted to <code>super_admin</code>
Notifications	<code>Notification</code>	<code>notificationController</code>	<code>notificationRoutes</code>	always scoped to <code>req.user._id</code> — never cross-user
Reports	(reads across models)	<code>reportController</code>	<code>reportRoutes</code>	same org-scoping, aggregated
Audit Log	<code>AuditLog</code>	(write-only via <code>auditLogger</code> util, read via future admin screen)	n/a yet	super_admin-only visibility (SRS §5.13)

4.3 Cross-Cutting Concerns

- **Error handling:** single `errorHandler` + `notFound` middleware pair, mounted last in `server.js`, so every controller can `throw new Error(...)` after setting `res.status(...)` and get a consistent JSON shape — this was hardened early (pulled forward from Step 21 into Step 5) once it became clear every later step's testing depends on clean error responses.
- **Auditability (SRS §5.13, §9.2):** any state-changing action on Partners, Admissions, and Commissions calls `logAction()` — this satisfies “every sensitive action must be recorded” without scattering logging logic per-controller.
- **Notifications:** fire-and-forget via `notify()` — a failure to create a notification never blocks the primary action (e.g. a lead reassignment still succeeds even if the notification write fails), matching the SRS's implicit priority (notifications are a UX nicety, not a transactional guarantee).

5. Frontend Design

5.1 Routing Structure (Next.js App Router)

□

```
/          → redirect based on auth state
/login     → public
/dashboard → protected (RequireAuth)
/dashboard/leads
/dashboard/followups
/dashboard/admissions
/dashboard/courses → protected + role-gated (super_admin)
/dashboard/partners → protected + role-gated (super_admin)
/dashboard/commissions
/dashboard/settings
```

This maps directly to the Handover Pack’s Screen List (§6) — every screen listed there has a corresponding route, except Campaign Center and full Marketing Asset Library UI, both explicitly deferred per §2 above.

5.2 Component Architecture

□

AuthProvider (context)

└ wraps entire app in layout.js — makes useAuth() available everywhere

RequireAuth (wrapper component)

└ used by every /dashboard/* page — redirects to /login if unauthenticated,
redirects to /dashboard if role-gated and role doesn't match

DashboardShell (layout component)

└ sidebar nav (role-filtered) + top bar — every dashboard page renders inside this,
so navigation is never duplicated per-page

Design rule enforced throughout: a page component only ever does two things — fetch/mutate data via `lib/api.js`, and render. Auth checks and navigation chrome are never re-implemented per page; they're inherited from `RequireAuth` + `DashboardShell`. This was a deliberate correction made in Step 10 (an earlier ad-hoc dashboard page was replaced) specifically to keep this rule intact before more pages were built on top of it.

5.3 State Management

No global state library (Redux/Zustand) — deliberately. The app's state needs are:

- **Auth state:** React Context (`AuthContext`) — read in a handful of places, changes rarely
- **Server data (leads, admissions, etc.):** fetched per-page with local `useState` + `useEffect`, no client-side cache layer

This matches the SRS's actual complexity (§9.5 scalability is about backend/DB horizontal scaling, not frontend state complexity) — introducing a state library would be premature engineering for the current scope. If report dashboards later need shared cross-page data, `TanStack Query` is the natural upgrade path (see §12 reference stack) rather than a rewrite.

5.4 API Client Design

`lib/api.js` centralizes:

- Base URL from `NEXT_PUBLIC_API_URL`
- Automatic `Authorization: Bearer <token>` header injection on every request

- Global 401 handling — clears session and redirects to `/login`, so no page needs its own "am I still logged in" boilerplate

6. Backend Design

6.1 Request Lifecycle



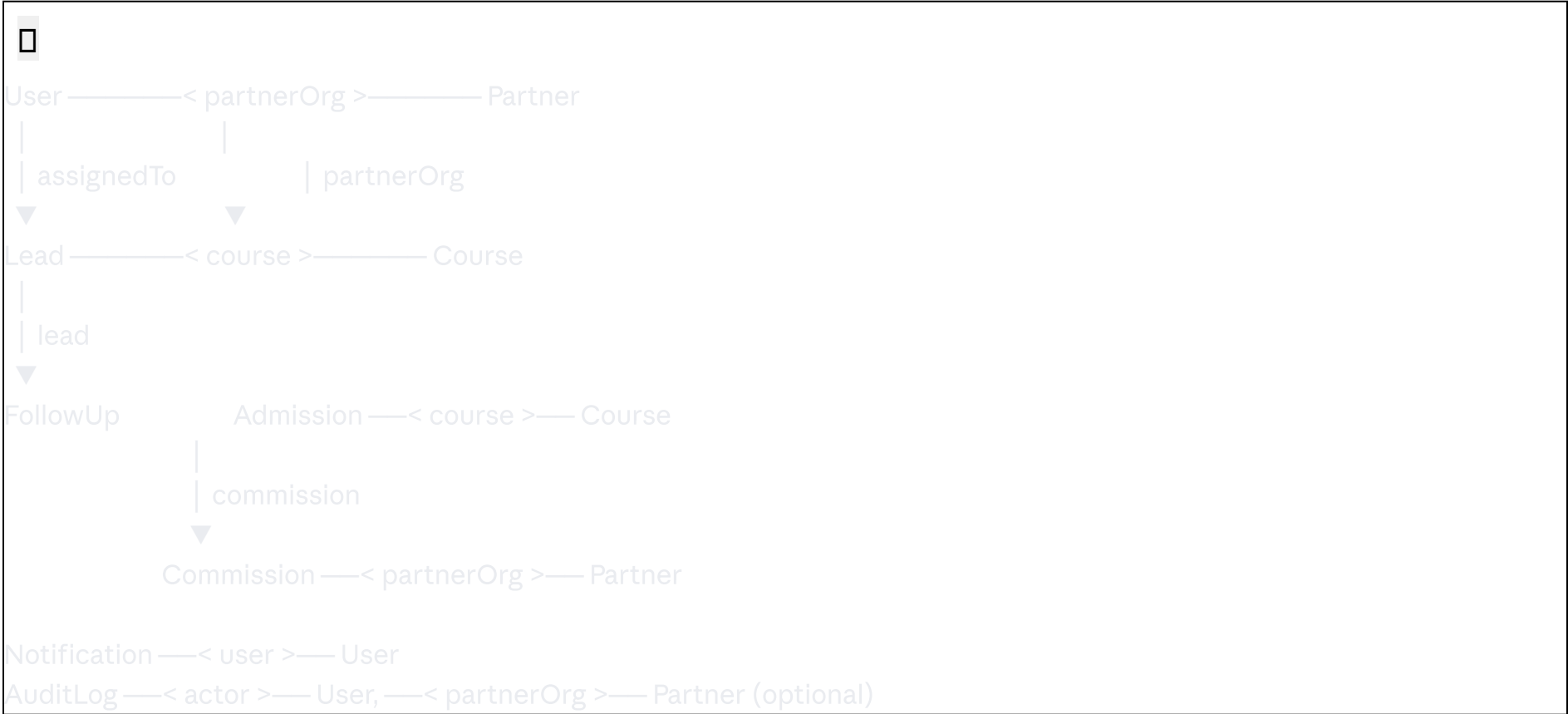
1. Client sends request with JWT in Authorization header
2. CORS middleware checks origin against CLIENT_URL
3. `express.json()` parses body
4. morgan logs the request (dev only)
5. Route matched → protect middleware:
 - a. Extracts + verifies JWT
 - b. Loads User from DB, checks `isActive`
 - c. Attaches `req.user`
6. `authorize(...roles)` middleware (if route is role-gated):
 - a. Checks `req.user.role` against allowed list
7. Controller function runs:
 - a. Builds role-scoped query filter
 - b. Performs business logic / validation
 - c. Calls Mongoose model methods
 - d. Optionally calls `logAction()` / `notify()`
8. Response sent as JSON
9. If any step throws → `errorHandler` middleware catches, returns `{ message, stack? }` with the status code the throw site set

These map directly to SRS §10 (Business Rules) and §12 (Acceptance Criteria):

SRS Business Rule	Enforced where	How
"A partner can access only its own tenant data"	Every controller's <code>buildScopeFilter</code>	Mongo query is pre-filtered by <code>partnerOrg</code> before any document is returned — not a post-fetch filter, so there's no path where cross-tenant data is even retrieved
"Duplicate leads should be flagged based on phone/email matching"	<code>leadController.createLead</code>	<code>normalizedPhone</code> (digits-only) computed via pre-save hook, checked against existing leads scoped to the same <code>partnerOrg</code> before insert; returns <code>409 Conflict</code> with the existing lead's id
"A lead cannot be marked admitted without an admission record"	<code>admissionController.createAdmission</code>	Lead's <code>status</code> only flips to <code>admission_confirmed</code> as a side effect of successfully creating an <code>Admission</code> document — there's no direct "set status to admission_confirmed" path on the Lead update endpoint
"Commission rules may differ by course or partner category"	<code>admissionController.updateAdmission</code> (commission auto-generation)	Rate resolution order: <code>course.commissionOverride</code> if set, else <code>partner.commissionRate</code> — course-level rule wins, satisfying "differ by course"
"Only authorized roles can view commission and partner management screens"	<code>partnerRoutes</code> , <code>commissionRoutes</code>	<code>authorize('super_admin')</code> on write paths; read paths still scoped by <code>buildScopeFilter</code> so partner_admin sees only their own commission data, never system-wide

7. Database Design

7.1 Entity Relationship Overview



7.2 Collection Schemas (Summary)

Collection	Key fields	Indexes	Notes
<code>users</code>	email (unique), password (hashed, <code>select:false</code>), role (enum), partnerOrg (ref), isActive	<code>email</code> unique	Password never returned by default queries

Collection	Key fields	Indexes	Notes
<code>partners</code>	name, contactEmail, status (enum: pending/approved/suspended), commissionRate, owner (ref User)	—	<code>status</code> enum matches SRS §5.2 (though SRS also lists "rejected" — see §2 reconciliation note below)
<code>courses</code>	title, category (enum), fee, commissionOverride (nullable), isActive, marketingAssets[]	—	Embedded array for marketing assets, per §2 decision
<code>leads</code>	fullName, phone, email, status (enum, 7 states), source (enum), partnerOrg (ref), assignedTo (ref), normalizedPhone	compound <code>{partnerOrg, normalizedPhone}</code>	Compound index makes duplicate-detection queries fast even at scale
<code>followups</code>	lead (ref), partnerOrg (ref), assignedTo (ref), scheduledDate, status (enum), outcome	<code>{assignedTo, status, scheduledDate}</code>	Index supports the "due today / overdue" dashboard queries directly
<code>admissions</code>	lead (ref), course (ref), partnerOrg (ref), admissionStatus (enum), totalFee, amountPaid, paymentStatus (auto-derived), commission (ref, nullable)	—	<code>paymentStatus</code> computed in pre-save hook from <code>amountPaid</code> vs <code>totalFee</code> — never set directly by client input
<code>commissions</code>	partnerOrg (ref), admission (ref), rateApplied, baseAmount, commissionAmount, payoutStatus (enum)	—	Immutable <code>rateApplied</code> / <code>baseAmount</code> snapshot at creation time — rate changes on the Partner/Course later don't retroactively alter past commissions
<code>notifications</code>	user (ref), title, message, type (enum), isRead	<code>{user, isRead, createdAt}</code>	Index supports "unread count" and "recent" queries efficiently

Collection	Key fields	Indexes	Notes
<code>auditlogs</code>	actor (ref), action (string), entityType, entityId, partnerOrg (ref, optional), metadata (mixed)	<code>{entityType, entityId}</code> , <code>{createdAt}</code>	<code>metadata</code> is intentionally schemaless (Mixed type) since different actions carry different useful context

Reconciliation note on Partner status: SRS §5.2 lists four values (`pending, active, suspended, rejected`); the implementation uses three (`pending, approved, suspended`). `approved` $\approx$ `active` (naming chosen to match the *action* — "approve a partner" — rather than the resulting state name, for controller readability: `updatePartnerStatus` reads naturally next to `status: 'approved'`). `rejected` was folded into re-using `suspended` rather than adding a fourth enum value, since the handover pack doesn't describe any distinct behavior for "rejected" vs "suspended" beyond the label. This is a minor, low-risk simplification — flagged here rather than silently diverging.

7.3 Why MongoDB Fits This Schema

- **Nullable, role-dependent fields** (`partnerOrg: null` for super_admin) are natural in a document model without needing a separate join table or nullable-FK gymnastics
- **Embedded** `marketingAssets[]` avoids a join for data that's always fetched together with its parent Course
- **Schemaless** `metadata` on **AuditLog** would require a JSON column + manual validation in a relational DB; it's native here
- **Read-heavy, tenant-scoped queries** (`{partnerOrg: X}`) are index-friendly in MongoDB and don't need cross-table joins the way a normalized relational schema would for the same access pattern

8. Core Workflows

8.1 Lead → Admission → Commission (SRS Use Case UC-02, UC-03; Handover §7)

□

▼
Notification sent to Partner.owner

8.2 Partner Onboarding (SRS UC-07; Handover §7)



Authenticated user (any role) submits Partner creation request



Partner.status = "pending", owner = requesting user



super_admin reviews → PUT /partners/:id/status { status: "approved" }



Partner.status = "approved"



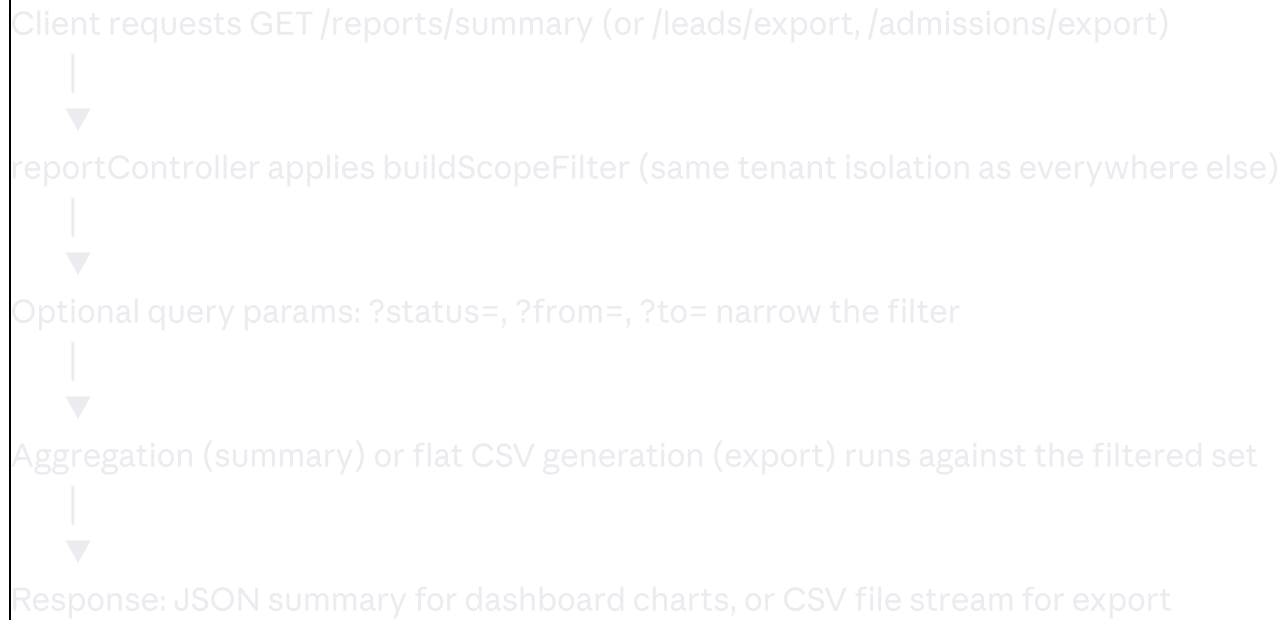
super_admin registers team_member/partner_admin users with
partnerOrg = that Partner's _id



Those users can now create Leads (blocked with 400 until partnerOrg is set —
enforced in leadController.createLead)

8.3 Reporting Flow (SRS UC-05, §5.11)





9. Security Design

Directly addressing SRS §9.2 and Handover §9:

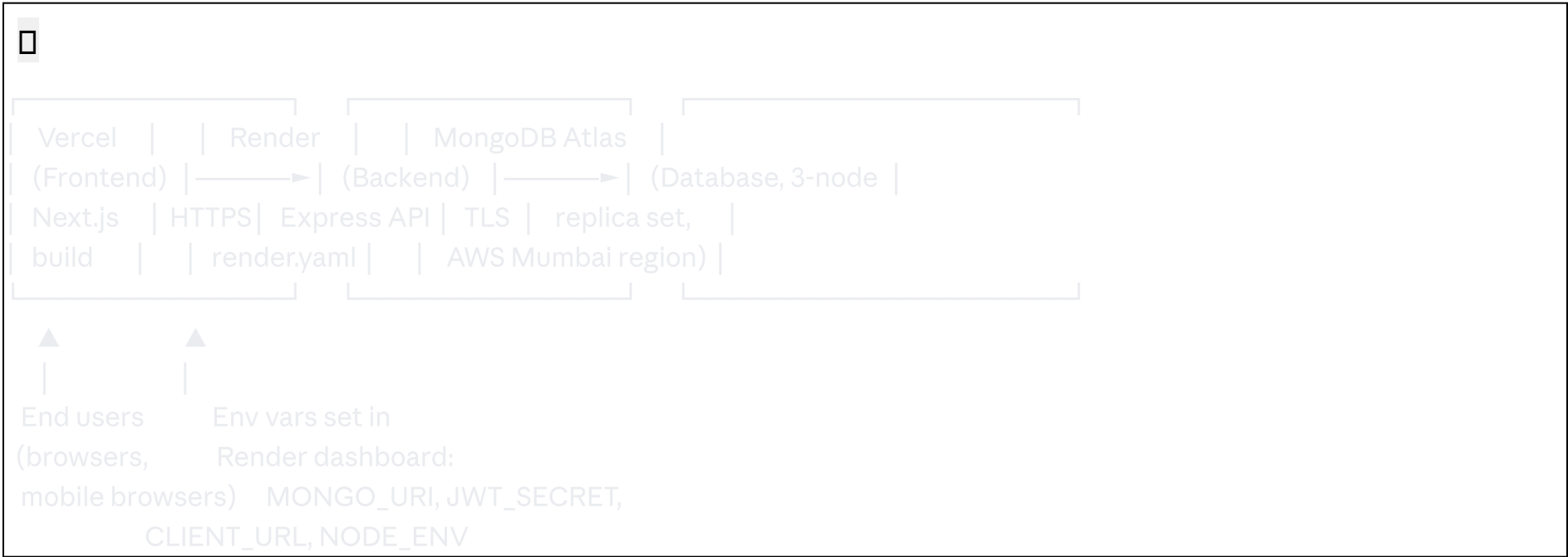
Requirement	Implementation
"Passwords shall be hashed securely"	<code>bcryptjs</code> , 10 salt rounds, applied in a Mongoose <code>pre('save')</code> hook — impossible to save a User document with a plaintext password by accident
"Role-based authorization shall protect all endpoints"	Every route module mounts <code>protect</code> via <code>router.use(protect)</code> ; write/admin routes additionally gate with <code>authorize(...)</code>
"Sensitive actions shall require audit logging"	<code>logAction()</code> called on Partner status changes, Admission updates, Commission auto-generation and payout changes
"Data export shall be controlled by permission"	Report export endpoints inherit the same <code>buildScopeFilter</code> as everything else — a <code>partner_admin</code> exporting CSV only ever gets their own org's rows
Session/token handling	Stateless JWT, 7-day expiry (configurable), never stored server-side — matches "record last login, device, session activity" only partially: session <i>activity</i> logging beyond login itself is not yet built (candidate for Step 20 follow-up if pursued further)
Rate limiting	<code>express-rate-limit</code> on <code>/api/auth/*</code> specifically — brute-force protection where it matters most, without throttling normal authenticated CRM usage
Transport security	HTTPS enforced at the hosting layer (Vercel/Render both terminate TLS); <code>CLIENT_URL</code> -restricted CORS prevents arbitrary origins from calling the API with a stolen token

Explicit gap vs. SRS: two-factor authentication (§5.1) and full session/device audit trail are both explicitly deferred by the SRS itself to "future releases" — not built, consistent with that framing.

10. Non-Functional Requirements Mapping

SRS §9 Requirement	Design response
9.1 Performance — fast dashboard load on mobile data	Indexes on the fields every dashboard query filters by (<code>partnerOrg</code> , <code>assignedTo</code> , <code>status</code> , <code>scheduledDate</code>); pagination available on Lead listing
9.1 Bulk operations processed asynchronously	Not yet needed at current scale (no bulk import built); flagged as the first place to introduce a job queue if lead-import volume grows
9.3 Reliability — data consistency on interruption	MongoDB Atlas replica set (3 nodes, confirmed in the actual cluster config) gives automatic failover; Mongoose write operations are not currently wrapped in multi-document transactions — acceptable at current scope since no single user action writes to more than 1–2 collections atomically-critically (e.g. commission creation is a direct consequence of a single admission update, not a distributed transaction)
9.4 Usability — simple for non-technical partners	Sidebar nav filtered by role (no "you can't use this" dead links for lower roles), consistent card/table visual language across every module
9.5 Scalability	Stateless backend (JWT, no server-side session store) means horizontal scaling is just "run more instances behind a load balancer" with no sticky-session requirement
9.6 Maintainability	Strict per-domain file structure (model/controller/route triplet) makes the codebase navigable without a map; <code>.env</code> -externalized config (DB URI, JWT secret, client URL)

11. Deployment Architecture



Config boundary discipline: `CLIENT_URL` (backend) and `NEXT_PUBLIC_API_URL` (frontend) are the only two values that need to change between local development and production — everything else in each `.env` is environment-specific but doesn't need cross-referencing the other service's deployed URL.

12. Documentation & Reference Stack

Curated from the full reference list supplied, narrowed to what's directly relevant to *this* project's actual stack (Node/Express/MongoDB/Next.js/Tailwind/JWT) rather than the full personal learning roadmap, which also covers Java/Spring and other stacks outside this project's scope.

Tier 1 — used directly in this build

Topic	Reference
JavaScript/DOM fundamentals	MDN Web Docs
Backend framework	Express.js Docs
ODM	Mongoose Docs
Database	MongoDB Documentation
Frontend framework	Next.js Documentation
UI library	React Docs
Styling	Tailwind CSS Documentation
Auth	JWT.io Introduction
HTTP client	Axios Docs

Tier 2 — hardening & production readiness (Steps 21-23)

Topic	Reference
Security checklist	OWASP Top 10
API-specific security	OWASP API Security Project
Rate limiting middleware	express-rate-limit Docs
Testing (backend)	Jest Documentation, Supertest

Topic**Reference**

Deployment — backend [Render Docs](#)

Deployment — frontend [Vercel Docs](#)

Database hosting [MongoDB Atlas Docs](#)

Tier 3 — architecture growth (post-MVP reference, not required for current scope)**Topic****Reference**

General system design [System Design Primer](#)

Refactoring guidance [Refactoring.Guru](#)

Query/state caching (candidate upgrade, §5.3) [TanStack Query Docs](#)

CI/CD (candidate for Step 23 expansion) [GitHub Actions Docs](#)

Git & workflow (used throughout)**Topic****Reference**

Git fundamentals [Pro Git Book](#)

Commit conventions [Conventional Commits](#)

13. Appendix — Phase Plan Alignment

Cross-referencing Handover Pack §14 (Phase Plan) against the 24-step build plan actually executed:

Handover Phase	Features	Build steps covering it
Phase 1 (Must-have)	Login, roles, dashboard, leads, follow-ups, admissions, reports	Steps 4–13, 18
Phase 2 (High)	Marketing center, commission module, notifications, export tools	Steps 14 (courses, incl. marketing assets), 16, 18 (export), 19
Phase 3 (Medium)	WhatsApp integration, payment gateway, automation, reminders	Not built — explicitly deferred, matches SRS §13/Handover §14 framing
Phase 4 (Future)	AI lead scoring, predictions, smart suggestions	Not built — explicitly deferred
Phase 5 (Advanced future)	LMS, quizzes, certificates, mobile app, franchise support	Not built — explicitly deferred

This confirms the 24-step build plan’s scope is a faithful implementation of Handover Phase 1 + Phase 2, with Phases 3–5 correctly left out rather than partially/inconsistently attempted.

End of document. This design doc is intended to be read alongside the build log (Steps 1–24) and the two source requirement documents — it does not replace either, it bridges them to the actual code.